FinanzApp

Informe del proyecto · Gestor de finanzas personales en Python + MySQL · 05/11/2026

FinanzApp es una aplicación de escritorio (CustomTkinter) con versión web para el celular (FastAPI). Registra gastos e ingresos, importa estados de cuenta, lee los avisos del banco que llegan al correo y muestra reportes. Está organizada en tres capas: **pantallas** (ui/, web/), **lógica** (utils/, email_parser/) y **datos** (database/, un archivo por tabla).

1. Qué se hizo

Cambio	Qué hace
Versión para celular	Página web (FastAPI) que se abre en el celular conectado al mismo WiFi. Usa la misma base de datos que la app de escritorio.
Recuperar contraseña	Código de 6 dígitos enviado por correo desde un Gmail de la app. Vence en 15 minutos, máximo 5 intentos y se guarda cifrado con bcrypt.
Correo bancario verificado	Se probó la conexión con Gmail y Yahoo y la lectura de avisos. Ahora avisa cuando llega un correo del banco cuyo asunto no coincide con el patrón.
Asistente (chatbot)	Botón «?» abajo a la derecha. Responde con datos reales (gastos, saldos, presupuestos) y explica cómo usar la app. Tiene botón «Limpiar».
Tutorial guiado	Aparece la primera vez que entra un usuario; se repite pidiéndoselo al asistente.
Adaptada a Argentina	Montos $ 1.250,50, bancos y comercios argentinos, calendario es_AR y CSV para el Excel argentino.
Transferencias	Pasar dinero entre cuentas sin que cuente como gasto ni ingreso.
Cuenta predeterminada	La que usa el gasto rápido y la que aparece elegida en los movimientos nuevos.
Git y GitHub	Repositorio con .gitignore que deja afuera contraseñas y datos personales.

Cómo se usa: **python src/main.py** abre la app de escritorio y **python src/main_web.py** la versión para el celular. Las tablas nuevas de la base de datos se crean y actualizan solas al iniciar.

2. Conexión del correo para recibir los avisos del banco

La app entra a tu buzón con el protocolo **IMAP** (el mismo que usan los programas de correo), busca los avisos de los bancos, saca el monto, la fecha y el comercio, y te los muestra para que confirmes antes de guardarlos. Nunca borra ni marca correos como leídos.

Cómo configurarlo

1. En tu cuenta de Google activa la **verificación en dos pasos** (myaccount.google.com → Seguridad).
2. Entra a **myaccount.google.com/apppasswords**, escribe un nombre (ej. «FinanzApp correo») y crea la contraseña. Google muestra 16 letras una sola vez: cópialas.
3. En la app: **Correo** → **Conectar correo**, elige Gmail, escribe tu correo y pega las 16 letras. La app prueba la conexión antes de guardar.
4. En «Bancos que se leen» enciende tu banco y toca **Sincronizar ahora**. Revisa la lista y toca **Guardar**.

Por qué una «contraseña de aplicación»: Gmail no deja entrar a programas con tu contraseña normal. Esa contraseña se guarda en el **llavero de Windows** (librería keyring), nunca en la base de datos ni en archivos.

Cómo funciona por dentro

Buzón (IMAP, puerto 993) → Buscar por remitente del banco → Bajar solo cabeceras → Bajar el correo si el asunto coincide → Regex: monto, fecha, comercio → Vista previa

1) Entrar al buzón. Se abre una conexión cifrada (SSL) y se inicia sesión con la contraseña del llavero.

`src/email_parser/lector_email.py`

```
SERVIDORES = {"gmail": "imap.gmail.com", "yahoo": "imap.mail.yahoo.com"}

def conectar(proveedor, email_conexion, password):
    servidor = imaplib.IMAP4_SSL(SERVIDORES[proveedor], 993, timeout=20)
    servidor.login(email_conexion, password)   # error si la contraseña está mal
    return servidor
```

2) Buscar solo correos de bancos. El propio servidor filtra por fecha y remitente, así no se descarga todo el buzón. Primero se bajan solo las cabeceras (rápido) y el correo completo únicamente si el asunto parece un aviso.

`lector_email.py · _leer_avisos()`

```
servidor.select("INBOX", readonly=True)          # solo lectura: no marca nada como leído
for plantilla in plantillas:                      # un banco = una plantilla
    criterio = ["SINCE", fecha_imap, "FROM", f'"{plantilla["remitente"]}"']
    _, datos = servidor.search(None, *criterio)   # ej. SINCE 06-Oct-2026 FROM "galicia"
    numeros.update(datos[0].split())
```

3) Sacar los datos con expresiones regulares (regex). Cada banco tiene una plantilla con patrones. Lo que queda entre paréntesis es lo que se guarda.

Patrón	Expresión	En «Consumo por $ 12.345,67 en Comercio: COTO»
Asunto	compra|consumo|pago|débito|...	«Consumo aprobado» → sí es aviso
Monto	(?:ARS|\$)\s*(\d[\d.,]*\d|\d)	12.345,67
Comercio	comercio\s*:?\s*(.{3,60}?) ...	COTO

lector_email.py · extraer()

```
def extraer(texto, asunto, plantilla, fecha_correo):
    # "12.345,67" -> 12345.67
    monto = convertir_monto(_grupo(plantilla["patron_monto"], texto))
    if not monto:
        return None             # se avisa "sin monto reconocible"
    fecha = convertir_fecha(_grupo(plantilla["patron_fecha"], texto)) or fecha_correo
    comercio = _grupo(plantilla["patron_comercio"], texto) or asunto
    # Ingreso si el asunto dice "acreditación", "recibiste", "haberes"...
    es_ingreso = any(p in normalizar(asunto) for p in PALABRAS_INGRESO)
    return {"fecha": fecha, "descripcion": comercio, "monto": monto,
            "tipo": "ingreso" if es_ingreso else "gasto"}
```

4) Sin congelar la ventana. Leer el correo tarda unos segundos, así que corre en un **hilo aparte** (threading) y la pantalla revisa cada 200 ms si ya terminó. Los correos ya guardados o descartados se anotan en la tabla **emails_procesados** para no mostrarlos dos veces.

Si un aviso no aparece: «no parecían avisos» = el asunto no coincide (agregar la palabra al patrón del asunto con el lápiz de editar); «sin monto reconocible» = el monto viene en otro formato.

3. El asistente (chatbot)

Es un chatbot de **respuestas programadas**: no usa inteligencia artificial ni internet, por eso es gratis y responde al instante. Busca palabras clave en la pregunta, elige el tema con más coincidencias y arma la respuesta; algunos temas leen los datos reales del usuario desde MySQL.

Pregunta: «¿Cuánto gasté?» → Normalizar: «cuanto gaste» → Puntaje por tema → Gana «gastos_mes» → Consulta a MySQL → Respuesta + botones

Paso 1: normalizar el texto

Se pasa a minúsculas y se quitan acentos y signos, para que «¿Cuánto GASTÉ?» y «cuanto gaste» sean iguales.

src/utils/asistente.py

```
def normalizar(texto):
    # quita acentos: "Cuánto" -> "Cuanto"
    texto = unicodedata.normalize("NFKD", texto).encode("ascii", "ignore").decode()
    return " ".join(texto.lower().replace("?", " ").replace("¿", " ").split())
```

Paso 2: los temas (intenciones)

Cada tema tiene una lista de frases clave y la función que responde. Hay 23 temas: gastos, ingresos, saldos, presupuestos, importar, correo, transferencias, recuperar contraseña, tutorial, etc.

asistente.py

```
INTENCIONES = [
    # (clave,        frases clave,                                    función que responde)
    ("gastos_mes",   ["cuanto gaste", "gaste", "mis gastos", ...],    _gastos_mes),
    ("saldos",       ["saldo", "cuanto tengo", "mis cuentas", ...],   _saldos),
    ("importar",     ["importar", "csv", "excel", "estado de cuenta"], _fija("Para importar: ...")),
    ...
]
```

Paso 3: elegir el tema con más puntos

Cada frase encontrada suma tantos puntos como palabras tiene, así «estado de cuenta» (3 puntos) gana a «cuenta» (1 punto). El **\b** exige que la frase empiece al inicio de una palabra: «tema» no cuenta dentro de «sistema».

asistente.py · responder()

```
def responder(pregunta, usuario):
    texto = normalizar(pregunta)
    mejor, puntos_mejor = None, 0
    for clave, frases, funcion in INTENCIONES:
        puntos = sum(len(f.split()) for f in frases
                     if re.search(r"\b" + re.escape(f), texto))
        if puntos > puntos_mejor:
            mejor, puntos_mejor = funcion, puntos
    if mejor is None:
        return _respuesta("No entendí bien. Prueba con otras palabras...",
                          sugerencias=SUGERENCIAS_INICIO)
    return mejor(usuario)
```

Paso 4: responder con datos reales

asistente.py · _gastos_mes()

```
def _gastos_mes(usuario):
    hoy, desde, hasta = _mes_actual()
    totales = transacciones_dao.totales(usuario["id"], desde, hasta)     # consulta a MySQL
    categorias = transacciones_dao.gastos_por_categoria(usuario["id"], desde, hasta)
    top = "\n".join(f"• {c['nombre']}: {dinero(c['total'])}" for c in categorias[:3])
    return _respuesta(f"Este mes llevas gastado {dinero(totales['gastos'])}.\n\n{top}",
                      acciones=[("Ver reportes", ("ir", "reportes"))])  # botón a Reportes
```

La pantalla del chat

El «cerebro» (asistente.py) no tiene nada de interfaz, por eso lo usan las dos versiones:

- **Escritorio** (ui/ventana_asistente.py): el botón «?» abre un panel con burbujas, atajos de preguntas, botones de acción y «Limpiar», que borra la conversación y vuelve a saludar.
- **Celular** (web/static/asistente.js): el botón «?» manda la pregunta al servidor con **fetch("/asistente")**, que llama al mismo responder() y devuelve JSON. La conversación se guarda en sessionStorage para no perderla al cambiar de pantalla.

src/ui/ventana_asistente.py

```
def enviar(self, texto=None):                          # al tocar Enviar o Enter
    texto = (texto or self.entrada.get()).strip()
    self._burbuja(texto, propia=True)                  # burbuja roja del usuario
    # burbuja gris con la respuesta + botones
    self._mostrar_respuesta(asistente.responder(texto, self.usuario))

def limpiar(self):                                     # botón «Limpiar»
    for hijo in self.conversacion.winfo_children():
        hijo.destroy()
    self._mostrar_respuesta(asistente.bienvenida(self.usuario))
```

Se probó con 30 preguntas distintas y acertó el tema en todas. Para agregar un tema nuevo basta con sumar una línea a INTENCIONES con sus frases clave y su respuesta.